

# GUIA OFICIAL DE DEMOSTRACION

TRABAJO FINAL DE INTEGRACIÓN | BSG INSTITUTE

|              |                                                                                                                         |
|--------------|-------------------------------------------------------------------------------------------------------------------------|
| ESTUDIANTE:  | Marcelo Gilardoni                                                                                                       |
| ROL:         | Solutions Architect / DevOps Engineer                                                                                   |
| REPOSITORIO: | <a href="https://github.com/gilardoni72/trading-roi-explainer">https://github.com/gilardoni72/trading-roi-explainer</a> |

## INDICE DE CONTENIDOS

|                                                                          |        |
|--------------------------------------------------------------------------|--------|
| 1. Inicio de la Aplicacion y Conexion de APIs                            | Pag. 2 |
| 2. Primera Prueba: Validacion de Seguridad Perimetral (HTTP 401)         | Pag. 2 |
| 3. Segunda Prueba: Simulacion Exitosa y Verificacion de Calculos         | Pag. 2 |
| 3.1 Verificacion de los Calculos Matematicos del Adaptador               | Pag. 3 |
| 3.2 Observabilidad de Latencia y Rendimiento                             | Pag. 3 |
| 4. Tercera Prueba: Simulacion de Falla de Consistencia (ACID - HTTP 503) | Pag. 3 |
| 4.1 Diagnostico de la Causa Fisica                                       | Pag. 3 |
| 4.2 Transaccionalidad Estricta y Consistencia (Garantia ACID)            | Pag. 4 |
| 4.3 Diagnostico Automatico en Consola (OpenTelemetry)                    | Pag. 4 |
| 5. Cuarta Prueba: Aseguramiento de Calidad y Latencias                   | Pag. 4 |
| 5.1 Pruebas Unitarias Automatizadas (Pytest)                             | Pag. 4 |
| 5.2 Benchmark de Rendimiento y Percentiles (p50 / p95)                   | Pag. 5 |

Usa este manual de forma secuencial durante la sustentacion en vivo.

## 1. Inicio de la Aplicacion y Conexion de APIs

Para arrancar tu servidor web asincrono, abre tu consola de PowerShell y ejecuta el comando literal:

```
& "C:\OpenCode\sesion_3\.venv\Scripts\uvicorn.exe" main:app --reload --host 127.0.0.1 --port 8000 --app-dir "C:\OpenCode\1-integracion-tbf\codigo"
```

Esta aplicacion se conecta de forma asincrona a dos APIs externas principales:

- CoinGecko API v3 (Conector de Precios): Obtiene el precio de cotizacion en tiempo real de Bitcoin, Ethereum y Solana sin requerir claves.
- Google Gemini API (IA Engine): El modelo gemini-3.5-flash redacta el informe analitico y los consejos de gestion de riesgo en espanol.

## 2. Primera Prueba: Validacion de Seguridad Perimetral (HTTP 401)

1. Abre en tu navegador la documentacion interactiva: <http://127.0.0.1:8000/docs>
2. Haz clic en 'Try it out' en el endpoint POST /api/v1/trading/explain.
3. Deja vacio el parametro 'X-API-Key' y presiona el boton azul 'Execute'.
4. Resultado Obtenido: El servidor bloquea de inmediato la llamada y retorna un codigo HTTP 401 Unauthorized, demostrando seguridad perimetral estricta.

## 3. Segunda Prueba: Simulacion Exitosa y Verificacion de Calculos

1. Ahora, ingresa en la casilla de la cabecera 'X-API-Key' la clave: 'trading-secret-key-123'
2. En el recuadro negro de la peticion (Request Body), copia y pega tu consulta:

```
{  "user_name": "Marcelo Gilardoni",  "message": "Tengo $1500 USD que me sobraron y quiero simular una inversion en BTC si sube a $80000"}
```
3. Presiona 'Execute' y observa la respuesta exitosa HTTP 200 OK con tu informe redactado por Gemini 3.5.

### 3.1 Verificación de los Cálculos Matemáticos del Adaptador

El sistema no alucina datos financieros, ya que el adaptador los calcula localmente antes de entregarlos al LLM:

- Deducción de Comisión (0.5%): Sobre un capital de \$1,500.00 USD, calcula de forma exacta: \$7.50 USD.
- Monto Neto Destinado a Compra: Saldo líquido libre para inversión: \$1,492.50 USD (\$1,500.00 - \$7.50).
- Consulta de Mercado Real: El conector asíncrono asocia el precio real de CoinGecko del instante de tu click.
- Cantidad de Activo Comprada: Se calcula la fracción exacta de Bitcoin adquirida: 0.019395 BTC (\$1492.50 / \$76,952.00).
- Simulación de Salida (ROI Proyectado): Proyecta ganancias si tocas tu meta de \$80,000 USD:
  - \* Valor de la Cartera: \$1,791.00 USD
  - \* Retorno Neto (Ganancia Líquida): \$291.00 USD
  - \* Retorno de Inversión (ROI): 19.40%

### 3.2 Observabilidad de Latencia y Rendimiento (x-process-time)

En las cabeceras de respuesta (Response headers), observa el valor de 'x-process-time'.

Un valor de ~886.90ms demuestra un rendimiento excelente. Significa que el ciclo completo (autenticación, enmascaramiento PII Marcelo Gilardoni -> M\*\*\*\*\* G\*\*\*\*\*, llamada asíncrona a CoinGecko, persistencia local y explicación con Gemini 3.5) toma menos de un segundo.

## 4. Tercera Prueba: Simulación de Falla de Consistencia (ACID - HTTP 503)

**Demuestra resiliencia y transaccionalidad robusta ante fallos físicos de disco o bases de datos:**

**1. Bloquear Base de Datos:** Ve a 'codigo/database/mock\_db.json' en Windows. Clic derecho -> Propiedades -> Marca la casilla 'Solo lectura' (Read-only) -> Aceptar.

**2. Ejecutar la Consulta:** Presiona 'Execute' de nuevo en tu navegador o ejecuta el script de consola:

```
python codigo/scripts/demo_gemini_streaming.py
```

**3. Resultado Obtenido:** Recibirás de manera limpia un código HTTP 503 Service Unavailable informando: 'Error de consistencia de datos: No se pudo registrar la simulación en base de datos. Motivo: Permission denied.'

#### 4.1 Diagnostico de la Causa Fisica

Al estar en 'Solo lectura', el sistema operativo de Windows le niega el permiso de escritura a Python, arrojando un IOError (Errno 13). Tu adaptador financiero intercepta el error al instante, abortando el proceso de forma segura.

#### 4.2 Transaccionalidad Estricta y Consistencia (Garantia ACID)

En ambientes financieros, la integridad es prioritaria. Si la base de datos local falla, abortamos la operacion inmediatamente antes de llamar a Gemini 3.5. Esto previene facturar costos de red inútiles e impide inconsistencias de datos, indicando de forma transparente al usuario que vuelva a intentar.

#### 4.3 Diagnostico Automatico en Consola (OpenTelemetry)

De fondo, en tu consola de PowerShell del servidor FastAPI, OpenTelemetry captura la excepcion PermissionError de forma autonoma, marcando el Span en estado 'ERROR' y asociando la traza, al mismo tiempo que mantiene tu servidor web levantado y asombrosamente resiliente ante otras peticiones.

### 5. Cuarta Prueba: Aseguramiento de Calidad y Latencias

Muestra al docente las evidencias cientificas y automatizadas de rendimiento de tu API:

A) Correr la Suite de Pruebas Unitarias de Pytest (Offline & Deterministas):

Comando literal: `python -m pytest codigo/tests/test_trading_api.py -v`

- Se ejecutaran 6 pruebas unitarias exitosas ('6 passed in ~1.05s') que verifican salud, claves, enmascaramiento y alias de compatibilidad en un entorno offline sin costos de API.

B) Correr el Benchmark de Latencias (10 llamadas consecutivas):

Comando literal: `python codigo/scripts/benchmark_latencia.py`

- Calcula los percentiles reales del servidor local: p50 (Mediana) de ~20.97 ms y p95 de ~28.86 ms, guardando las mediciones de rendimiento en el archivo 'database/benchmark\_results.json'.